

Day 5: Production Systems

Agentic AI Intensive Training

5-Day Program

Today's Agenda

Morning: Production Readiness

- Architecture patterns
- Deployment strategies
- Scaling considerations

Afternoon: Operations

- Monitoring & observability
- Security & compliance
- Cost optimization

Production vs Prototype

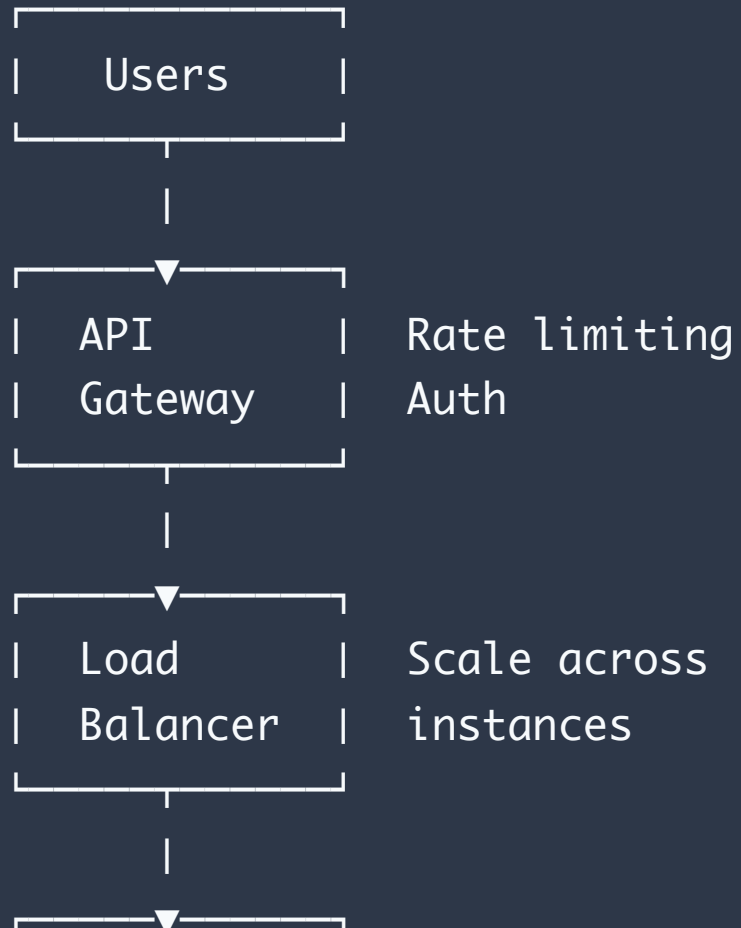
Prototype

- Proof of concept
- Limited users
- Manual intervention OK
- Cost not critical

Production

- Reliable & scalable
- Many users

Production Architecture



System Design Principles

Reliability

- Graceful degradation
- Retry logic
- Circuit breakers
- Fallback responses

Scalability

- Horizontal scaling
- Async processing

API Design

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class QueryRequest(BaseModel):
    query: str
    max_tokens: int = 1000
    temperature: float = 0.7
```

```
class QueryResponse(BaseModel):
    answer: str
    sources: list[str]
    tokens_used: int
```

TypeScript API Design

```
import express from 'express';
import { z } from 'zod';

const app = express();

const QuerySchema = z.object({
  query: z.string(),
  maxTokens: z.number().default(1000),
  temperature: z.number().default(0.7)
});

app.post('/query', async (req, res) => {
  try {
    const { query, maxTokens, temperature } =
      QuerySchema.parse(req.body);
```

Error Handling

```
class AIServiceError(Exception):
    """Base exception for AI service"""
    pass

class RateLimitError(AIServiceError):
    """Rate limit exceeded"""
    pass

class ModelError(AIServiceError):
    """Model inference error"""
    pass

async def safe_llm_call(prompt, max_retries=3):
    for attempt in range(max_retries):
        try:
```


Rate Limiting

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@app.post("/query")
@limiter.limit("10/minute") # 10 requests per minute
async def query_endpoint(request: Request, data: QueryRequest):
    result = await rag_pipeline(data.query)
    return result

# Per-user limits
@limiter.limit("100/hour", key_func=lambda: get_user_id())
async def user_endpoint(request: Request):
    pass
```

Caching Strategy

```
from functools import lru_cache
import redis

# In-memory cache
@lru_cache(maxsize=1000)
def get_embedding_cached(text: str):
    return get_embedding(text)

# Redis cache
redis_client = redis.Redis(host='localhost', port=6379)

def cached_llm_call(prompt: str, ttl=3600):
    cache_key = f"llm:{hash(prompt)}"

    # Check cache
```

Async Processing

```
from celery import Celery

celery_app = Celery('tasks', broker='redis://localhost:6379')

@celery_app.task
def process_long_query(query_id: str, query: str):
    """Process query asynchronously"""
    try:
        result = rag_pipeline(query)

        # Store result
        store_result(query_id, result)

        # Notify user
        notify_completion(query_id)
```

Streaming Responses

```
from fastapi.responses import StreamingResponse

async def stream_generator(query: str):
    """Stream LLM response"""
    async for chunk in llm_stream(query):
        yield f"data: {json.dumps({'text': chunk})}\n\n"

@app.post("/query/stream")
async def stream_query(request: QueryRequest):
    return StreamingResponse(
        stream_generator(request.query),
        media_type="text/event-stream"
    )

# Client consumption
```

Database Design

```
from sqlalchemy import Column, Integer, String, Text, DateTime
from sqlalchemy.ext.declarative import declarative_base
```

```
Base = declarative_base()
```

```
class Query(Base):
    __tablename__ = 'queries'

    id = Column(Integer, primary_key=True)
    user_id = Column(String(50), index=True)
    query_text = Column(Text)
    response = Column(Text)
    tokens_used = Column(Integer)
    latency_ms = Column(Integer)
    timestamp = Column(DateTime, index=True)
```

Logging Best Practices

```
import logging
import structlog

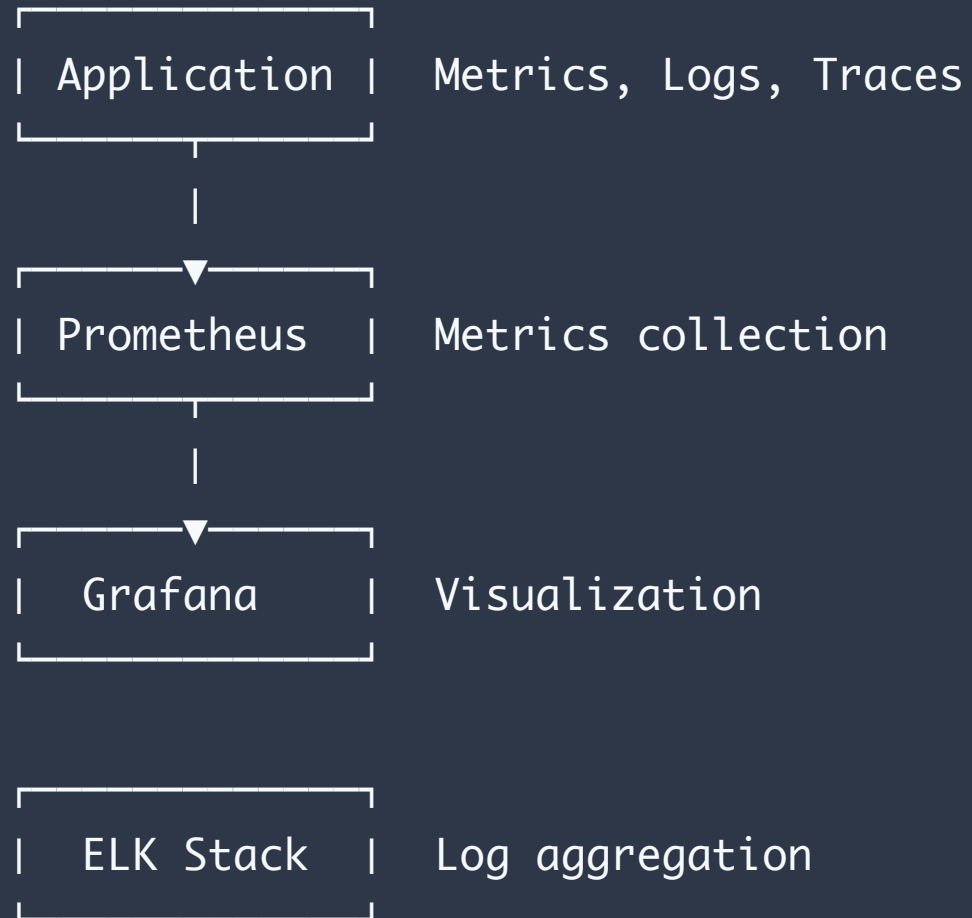
# Structured logging
logger = structlog.get_logger()

def process_query(query: str, user_id: str):
    log = logger.bind(
        user_id=user_id,
        query_length=len(query)
    )

    log.info("query_started")

    +rv.
```

Monitoring Stack



Metrics Collection

```
from prometheus_client import Counter, Histogram, Gauge

# Define metrics
request_count = Counter(
    'rag_requests_total',
    'Total RAG requests',
    ['endpoint', 'status']
)

request_duration = Histogram(
    'rag_request_duration_seconds',
    'Request duration',
    ['endpoint']
)
```


Key Metrics to Track

Performance

- Request latency (p50, p95, p99)
- Token usage (avg, max)
- Cache hit rate
- Throughput (requests/sec)

Quality

- User satisfaction ratings
- Retry rate

Alerting Rules

```
# Prometheus alert rules
groups:
  - name: rag_service
    interval: 30s
    rules:
      - alert: HighErrorRate
        expr: rate(rag_requests_total{status="error"}[5m]) > 0.05
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High error rate detected"

      - alert: HighLatency
        expr: histogram_quantile(0.95, rag_request_duration_seconds) > 5
```

Security: Authentication

```
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt

security = HTTPBearer()

def verify_token(
    credentials: HTTPAuthorizationCredentials = Depends(security)
) -> dict:
    try:
        token = credentials.credentials
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return payload
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")
```

Security: Input Validation

```
from pydantic import BaseModel, validator

class QueryRequest(BaseModel):
    query: str

    @validator('query')
    def validate_query(cls, v):
        # Length check
        if len(v) > 10000:
            raise ValueError("Query too long")

        # Character validation
        if not v.strip():
            raise ValueError("Query cannot be empty")
```

Security: Prompt Injection

```
def sanitize_prompt(user_input: str) -> str:
    """Prevent prompt injection"""
    # Remove control sequences
    sanitized = user_input.replace('\x00', '')

    # Escape special tokens
    sanitized = sanitized.replace('<lendoftext|>', '')

    return sanitized

def safe_prompt_construction(user_query: str) -> str:
    """Construct prompt safely"""
    sanitized_query = sanitize_prompt(user_query)

    prompt = f"""
```

Data Privacy

PII Handling

```
import re

def detect_pii(text: str) -> list[str]:
    """Detect potential PII"""
    patterns = {
        'email': r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',
        'phone': r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b',
        'ssn': r'\b\d{3}-\d{2}-\d{4}\b',
        'credit_card': r'\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?\d{4}\b'
    }

    detected = []
    for pii_type, pattern in patterns.items():
```

Compliance: GDPR

Data Subject Rights

```
class GDPRCompliance:
    def __init__(self, db):
        self.db = db

    def export_user_data(self, user_id: str) -> dict:
        """Right to data portability"""
        queries = self.db.get_user_queries(user_id)
        return {
            "user_id": user_id,
            "queries": queries,
            "export_date": datetime.utcnow()
        }
```

Cost Optimization

Strategies

Strategy	Savings	Tradeoff
Smaller models	50-90%	Accuracy
Caching	30-70%	Freshness
Batching	20-40%	Latency
Prompt compression	10-30%	Context
Smart routing	20-50%	Complexity

Cost Tracking

```
class CostTracker:
    def __init__(self):
        self.costs = {
            "gpt-4o": {"input": 0.005, "output": 0.015}, # per 1K tokens
            "gpt-4o-mini": {"input": 0.00015, "output": 0.0006},
            "embeddings": 0.00013
        }

    def calculate_cost(self, model: str, input_tokens: int,
                      output_tokens: int) -> float:
        rates = self.costs[model]
        cost = (
            (input_tokens / 1000) * rates["input"] +
            (output_tokens / 1000) * rates["output"]
        )
```

Smart Model Routing

```
def route_to_model(query: str, complexity: str = None) -> str:
    """Route based on query complexity"""
    if complexity is None:
        complexity = assess_complexity(query)

    if complexity == "simple":
        return "gpt-4o-mini" # Cheaper, faster
    elif complexity == "medium":
        return "gpt-4o-mini"
    else:
        return "gpt-4o" # More capable

def assess_complexity(query: str) -> str:
    """Assess query complexity"""
    # Simple heuristics
```

Batch Processing

```
async def batch_embed(texts: list[str], batch_size: int = 100):  
    """Batch embedding generation"""  
    embeddings = []  
  
    for i in range(0, len(texts), batch_size):  
        batch = texts[i:i + batch_size]  
  
        response = await client.embeddings.create(  
            input=batch,  
            model="text-embedding-3-small"  
        )  
  
        batch_embeddings = [d.embedding for d in response.data]  
        embeddings.extend(batch_embeddings)
```

Deployment: Docker

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Deployment: Kubernetes

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rag-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: rag-service
  template:
    metadata:
      labels:
        app: rag-service
    spec:
```

Horizontal Scaling

```
# HorizontalPodAutoscaler
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: rag-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: rag-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
```

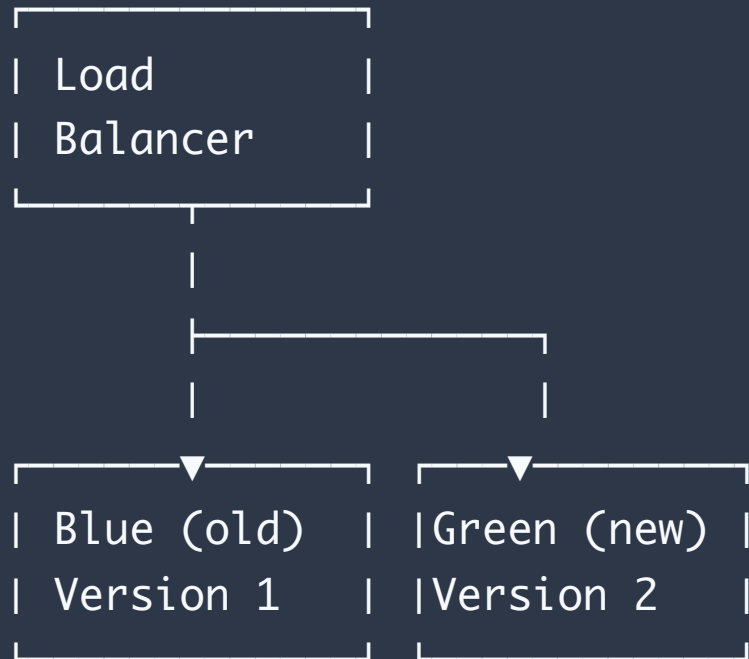
CI/CD Pipeline

```
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run tests
        run: |
          pip install -r requirements.txt
```

Blue-Green Deployment



1. Deploy green
2. Test green
3. Switch traffic

Feature Flags

```
from flagsmith import Flagsmith

flagsmith = Flagsmith(environment_key="...")

def get_model_for_user(user_id: str) -> str:
    flags = flagsmith.get_identity_flags(user_id)

    # Check feature flag
    use_new_model = flags.is_feature_enabled("new_model_rollout")

    if use_new_model:
        return "gpt-4o"
    else:
        return "gpt-4o-mini"
```

Load Testing

```
# locustfile.py
from locust import HttpUser, task, between

class RAGUser(HttpUser):
    wait_time = between(1, 3)

    @task
    def query(self):
        self.client.post("/query", json={
            "query": "What is machine learning?",
            "max_tokens": 1000
        })

    @task(2)
    def health_check(self):
```

Chaos Engineering

```
import random

class ChaosMiddleware:
    """Simulate failures for testing"""
    def __init__(self, failure_rate=0.1):
        self.failure_rate = failure_rate

    async def __call__(self, request, call_next):
        # Randomly inject failures
        if random.random() < self.failure_rate:
            raise HTTPException(status_code=503, detail="Chaos!")

        # Random latency
        if random.random() < 0.2:
            await asyncio.sleep(random.uniform(0.5, 2.0))
```

Incident Response

Playbook

1. **Detection:** Alert fires
2. **Triage:** Assess severity
3. **Mitigation:** Quick fix
4. **Communication:** Update stakeholders
5. **Investigation:** Root cause analysis
6. **Resolution:** Permanent fix
7. **Postmortem:** Document learnings

Postmortem Template

Incident Postmortem: [Date]

Summary

Brief description of the incident.

Timeline

- 14:00 - Alert fired: High error rate
- 14:05 - Engineer paged
- 14:15 - Root cause identified
- 14:30 - Fix deployed
- 14:45 - Service recovered

Root Cause

Vector DB connection pool exhausted.

Observability: Tracing

```
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

tracer = trace.get_tracer(__name__)

@app.post("/query")
async def query_endpoint(request: QueryRequest):
    with tracer.start_as_current_span("query_endpoint"):
        # Retrieval
        with tracer.start_as_current_span("retrieval"):
            docs = await retrieve_context(request.query)

        # Generation
        with tracer.start_as_current_span("generation"):
            answer = await generate_answer(request.query, docs)
```

Documentation: API

```
from fastapi import FastAPI

app = FastAPI(
    title="RAG Service API",
    description="Retrieval-Augmented Generation service",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc"
)

@app.post(
    "/query",
    summary="Query the RAG system",
    description="Submit a query and receive an AI-generated answer",
    response_description="Answer with sources"
```

Documentation: Architecture

RAG Service Architecture

Components

API Layer

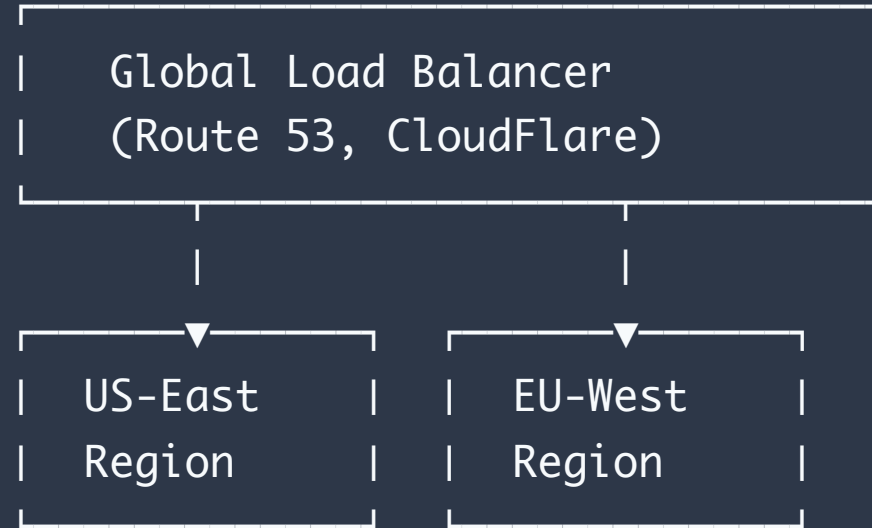
- FastAPI application
- Rate limiting: 100 req/min per user
- Authentication: JWT tokens

Processing Layer

- Retrieval: Pinecone vector DB
- Generation: OpenAI GPT-4
- Caching: Redis

Data Layer

Multi-Region Deployment



Benefits:

- Lower latency
- Higher availability
- Data residency compliance

Disaster Recovery

```
class DisasterRecovery:
    def __init__(self):
        self.primary_db = "us-east-1"
        self.backup_db = "us-west-2"

    async def query_with_failover(self, query: str):
        try:
            # Try primary
            return await self.primary_db.query(query)
        except Exception as e:
            logger.error(f"Primary failed: {e}")

            # Fallback to backup
            try:
                return await self.backup_db.query(query)
```

Case Study: Customer Support

Problem: 10k+ support tickets/month

Solution: RAG-based assistant

- Knowledge base: docs, past tickets
- Retrieval: top 5 relevant articles
- Generation: suggested responses

Results

- 60% tickets auto-resolved

Case Study: Legal Research

Problem: Manual case law research (hours)

Solution: Legal RAG system

- Corpus: 1M+ court cases
- Specialized embeddings: legal domain
- Multi-step reasoning: complex queries

Results

- 10x faster research

Case Study: Code Assistant

Problem: Developer documentation scattered

Solution: Code-aware RAG

- Index: API docs, code repos, Stack Overflow
- Code understanding: AST parsing
- Multi-modal: code + natural language

Results

- 50% faster onboarding

Industry Trends

Current

- Larger context windows (1M+ tokens)
- Multi-modal models (text, image, audio)
- Faster inference (Groq, Fireworks)
- Open source momentum (Llama, Mistral)

Coming Soon

- Reasoning models (o1-style)
- Agentic workflows

Responsible AI & Governance

Beyond security, ensure systems are **ethical, accountable, and trustworthy**.

Responsible AI Framework

Four Pillars:

1. **Ethics & Boundaries** - When (not) to automate
2. **Fairness & Bias** - Detect and mitigate bias
3. **Explainability** - Make decisions transparent
4. **Governance** - Safe deployment practices

When NOT to Automate

Never Fully Automate:

- Legal decisions
- Medical diagnoses
- Financial transactions
- Safety-critical systems
- Irreversible actions

Require Human Approval:

- Production deployments

Bias Detection

```
class BiasDetector:
    def detect_bias(self, text: str) -> Dict[str, Any]:
        """Detect potential bias in AI outputs."""
        results = {
            "has_bias": False,
            "bias_types": [],
            "suggestions": []
        }

        # Check gender bias
        if self._check_gender_bias(text) > 0.3:
            results["bias_types"].append("gender")
            results["suggestions"].append(
                "Use gender-neutral language"
            )
```

Fairness Testing

Test AI performance across different user groups:

```
class FairnessTester:
    def test_fairness(
        self,
        predictions: List[Dict],
        protected_attribute: str
    ):
        """Test for disparate impact."""
        # Group by attribute
        groups = self._group_predictions(predictions)

        # Calculate metrics per group
        metrics = self._calculate_metrics(groups)
```

Human-in-the-Loop

Confidence-Based Escalation:

- High confidence (>90%): Auto-approve
- Medium confidence (50-90%): Human review
- Low confidence (<50%): Auto-reject

```
if confidence >= 0.9:  
    # Proceed automatically  
    execute_action()  
elif confidence >= 0.5:  
    # Escalate to human  
    await request_approval()
```

Audit Trails

Log every AI decision for compliance:

```
audit.log_decision(  
    action_type="code_review",  
    input_data={"pr": 456},  
    output_data={"approved": True},  
    model="claude-3-5-sonnet",  
    confidence=0.89,  
    reasoning="Tests passed, no vulnerabilities",  
    approved_by="engineer@company.com",  
    cost=0.002  
)
```

Essential for:

Explainability

Make AI decisions understandable:

```
explanation = explainer.explain_decision(  
    decision="Approve PR #456",  
    confidence=0.87,  
    reasoning="Well-tested, follows best practices"  
)  
  
# Returns:  
{  
    "summary": "AI is confident (87%) that...",  
    "key_factors": ["All tests pass", "No vulnerabilities"],  
    "confidence_level": "High - some uncertainty remains",  
    "human_oversight": "Medium - review before implementation"  
}
```

Phased Rollout

Safe deployment strategy:

Phases:

1. **Canary** (5%) - Initial testing
2. **Small** (25%) - Expanded testing
3. **Medium** (50%) - Half of users
4. **Large** (75%) - Majority
5. **Full** (100%) - Complete rollout

Monitor at each phase:

Kill Switch

Emergency stop for AI systems:

```
kill_switch = KillSwitch()

# Monitor for anomalies
if error_rate > 0.15:
    kill_switch.disable(
        reason="Error rate exceeded threshold",
        disabled_by="monitoring-system"
    )

# All subsequent requests blocked
# Manual re-enable required
```


Responsible AI Checklist

Before Production:

- ☐ Automation decision framework documented
- ☐ Bias testing completed
- ☐ Human-in-the-loop for high-risk actions
- ☐ Audit trail implemented
- ☐ Explainability mechanisms in place
- ☐ Phased rollout plan ready
- ☐ Kill switch implemented

Key Ethical Issues

Be Aware:

- Bias in training data
- Misinformation/hallucinations
- Privacy concerns
- Job displacement
- Environmental impact

Best Practices:

- Diverse training data

Final Workshop: End-to-End

Build: Production-ready RAG service

Requirements

- FastAPI endpoint
- Vector DB integration
- Monitoring/logging
- Error handling
- Tests
- Docker deployment

Workshop Checklist

- ☐ API with authentication
- ☐ RAG pipeline implementation
- ☐ Caching layer
- ☐ Rate limiting
- ☐ Error handling
- ☐ Logging with structured logs
- ☐ Metrics (Prometheus)
- ☐ Unit tests

Graduation Requirements

Completed

- All 5 day workshops
- Final project: Production RAG service
- Code review passed
- Presentation of your system

Certificate

Agentic AI Intensive Training Program

Next Steps

Course Recap

Day 1: Foundations & LLM basics

Day 2: Advanced prompting techniques

Day 3: Agentic systems

Day 4: RAG & evaluation

Day 5: Production deployment

You now know

- How LLMs work
- Prompt engineering
- Building agents

Final Advice

Keep Building

- Best way to learn
- Start small, iterate
- Share your work

Stay Current

- Field evolves rapidly
- Follow researchers
- Try new models/techniques

Thank You

Congratulations on completing the program!

Keep Learning

The journey is just beginning.

Q&A

Final questions?

Resources shared

- Slide deck
- Code examples
- Project templates
- Reading list
- Community links

Farewell

You're now ready to build production AI systems.

Go build something amazing!

See you in the AI community.